DELIVERABLE 1

Data Exploration, Visualisation & Pre-processing Report

Dataset B: PlantVillage — Plant-Disease Classification

Plant Recognition · Capstone Project

Exploration, visualisation and data audit of the raw PlantVillage image set (38 crop / disease classes), and the pre-processing that turns it into a dataset ready for deep-learning modelling.

Cohort may26bds_int

Prepared by the project team

Adrian Bogdan RUS · Alexander DANZ · Christoph Clemens Martin NEDDENS · Scheima Sara OBEIDI

Project mentor: Habiba El Husseiny (ML Engineer)

Contents

1. Introduction

This report is Deliverable 1 of the Plant Recognition capstone project. It brings together the Step 1 data exploration and the Step 2 pre-processing for the disease-classification half of the project, and its purpose is to document, with evidence, how the raw PlantVillage image set was understood and then turned into a dataset ready for deep-learning modelling.

Plant Recognition builds two independent image classifiers — one that identifies a plant's species and one that identifies its disease. This report concerns the second, trained on the PlantVillage dataset. From a business point of view the aim is an accessible tool, in the spirit of applications such as PlantNet, that takes a photograph of a leaf and returns the crop and its likely disease, helping growers act earlier and reducing crop loss. Technically the task is fine-grained image classification, addressed with a convolutional neural network; economically its value lies in cheaper, faster diagnosis than expert inspection; scientifically it is an exercise in separating visually similar categories where the discriminating signal is small and local.

The work proceeds in two stages that this document mirrors. Step 1 (Sections 2–6) explores the data thoroughly enough that every later choice rests on evidence rather than assumption. Step 2 (Section 7) cleans and pre-processes the data into modelling-ready form. Section 8 draws the two together into the implications for modelling.

The team comprises four data-science students, each owning part of the pipeline — image loading and normalisation, dataset splitting, and augmentation and class balancing — with a project mentor providing guidance. No external business experts were consulted; the reference application PlantNet and the cited literature informed the framing.

In brief, the exploration shows the data to be large and visually uniform but heavily imbalanced; its crop and disease labels are entangled; and — the key finding — a leaf's overall colour barely distinguishes healthy from diseased, because the disease signal is local. These findings drive a pre-processing pipeline built around a stratified, leakage-free split, normalisation applied at the model input, on-the-fly augmentation, and class-imbalance correction deferred to training.

2. The dataset and its structure

The dataset is PlantVillage, obtained from Kaggle in its faithful, un-modified form (the abdallahalidev mirror). It is freely and publicly available under its Kaggle licence. Using the raw images rather than a pre-split or pre-augmented repackaging is a deliberate choice: it means every transformation applied later — the train/validation/test split, any augmentation, the class balancing — is one this project controls and can account for, which makes the pipeline reproducible and defensible.

Each image is labelled by the folder it sits in, with a two-part name of the form crop___condition — for example, Tomato___Late_blight or Apple___healthy. Parsing these labels yields 14 distinct crops and 21 distinct conditions (“healthy” plus twenty named diseases), combining into 38 classes in total. The condition reduces naturally to a healthy-or-diseased status, the axis at the heart of the task.

The downloaded archive wraps three parallel renderings of every photograph — color (the real RGB images), grayscale, and segmented (background removed). An integrity check confirms all three contain the same 38 classes with matching image counts, so they are the same pictures in different dress rather than additional data. The exploration and pre-processing therefore use the color rendering; grayscale would discard colour a disease model may use, while the segmented rendering is kept in reserve as a possible aid against background bias. Restricted to color, the working set is 54,305 images, each a uniform 256×256-pixel, three-channel square.

3. How the data was explored

Images cannot be analysed directly with ordinary table tools, so the exploration first builds a metadata table: one row per image recording its measurable properties — width, height, pixel count, channel count, file size, overall brightness, and the mean of its red, green and blue channels — alongside its label and the crop, condition and healthy/diseased status parsed from it.

Two passes are used. The exact class distribution comes from a full count of every file, which is fast because it opens no images. The pixel-level features come from a random sample of up to 250 images per class (9,402 images in total; one class, healthy Potato, has only 152 to draw from), cached to disk so the scan runs only once. Sampling the same number from every class keeps even the rarest classes represented.

Following the project methodology, each figure is read in two parts: a plain reading of what it means for the project, and a statistical validation that confirms the visual impression is real. The tests used are the chi-square goodness-of-fit and chi-square test of independence for the categorical questions, a one-way ANOVA with an effect-size measure for the colour comparison, and Pearson correlation for feature redundancy.

4. Data-audit table

The data-audit records one row for each of the metadata table's fifteen variables: its description, storage type, share of missing values, whether it is categorical or quantitative, a short summary of its distribution, and a comment on modelling relevance. Class-count distributions are taken from the full data; pixel-feature distributions from the 250-per-class sample. The notebook also exports this table as DATA_AUDIT_plantvillage.xlsx.

#	Column	Description	Type	% miss.	Cat./Quant.	Distribution	Comments
1	filepath	Absolute path to the image; unique id.	object	0.0%	Identifier	9,402 sampled paths.	Not predictive; used to load the image.
2	label	Full class crop___condition — the 38-class disease target.	object	0.0%	Categorical	38 classes; 5,507 (max) to 152 (min).	Extreme imbalance ~36:1. Predict this (or crop + condition).
3	crop	Plant type parsed from the label.	object	0.0%	Categorical	14 crops; Tomato is the largest.	Entangled with disease; several crops single-condition.
4	condition	Specific condition / disease parsed from the label.	object	0.0%	Categorical	21 distinct conditions.	Fine-grained disease label.
5	is_healthy	True when the condition is exactly 'healthy'.	bool	0.0%	Boolean	31% healthy (sample).	Core healthy/diseased axis; only for crops with both.
6	width	Image width in pixels.	int64	0.0%	Quantitative	constant 256 px.	Uniform → no resizing needed for consistency.
7	height	Image height in pixels.	int64	0.0%	Quantitative	constant 256 px.	Uniform; images are 256×256.
8	aspect_ratio	Width / height.	float64	0.0%	Quantitative	constant 1.	Square → no distortion from a square resize.
9	n_pixels	Width × height (resolution).	int64	0.0%	Quantitative	constant 65,536.	Uniform resolution.
10	channels	Number of colour channels.	int64	0.0%	Categorical	constant 3 (RGB).	Already standardized to 3 channels.
11	file_size_kb	File size on disk (KB).	float64	0.0%	Quantitative	min 5, median 16, max 30 KB.	On fixed-size JPEGs, larger ≈ more detail; weak signal.
12	brightness	Mean pixel intensity (0–255).	float64	0.0%	Quantitative	mean 118 of 255.	Controlled lab lighting; fairly uniform.
13	mean_R	Mean of the red channel.	float64	0.0%	Quantitative	mean 120.	Colour highly redundant with brightness.
14	mean_G	Mean of the green channel.	float64	0.0%	Quantitative	mean 126.	Dominant channel (foliage).
15	mean_B	Mean of the blue channel.	float64	0.0%	Quantitative	mean 108.	Lowest channel.

Reading the audit. Three things stand out. No variable has any missing values, so the data is complete and needs no imputation. The five geometry variables (width, height, aspect ratio, pixel count, channels) are constant, confirming the images are uniform and that resizing for consistency is unnecessary. And the target, label, is extremely imbalanced at about 36:1, while the colour variables are visibly redundant (green is the dominant channel and all three track overall brightness) — foreshadowing two of the findings below.

5. Findings: visualisations and statistical tests

The five figures below each pair a chart with a statistical test. Every figure is followed by what it shows for the project and the statistical result that validates it.

Figure 1 — Class distribution

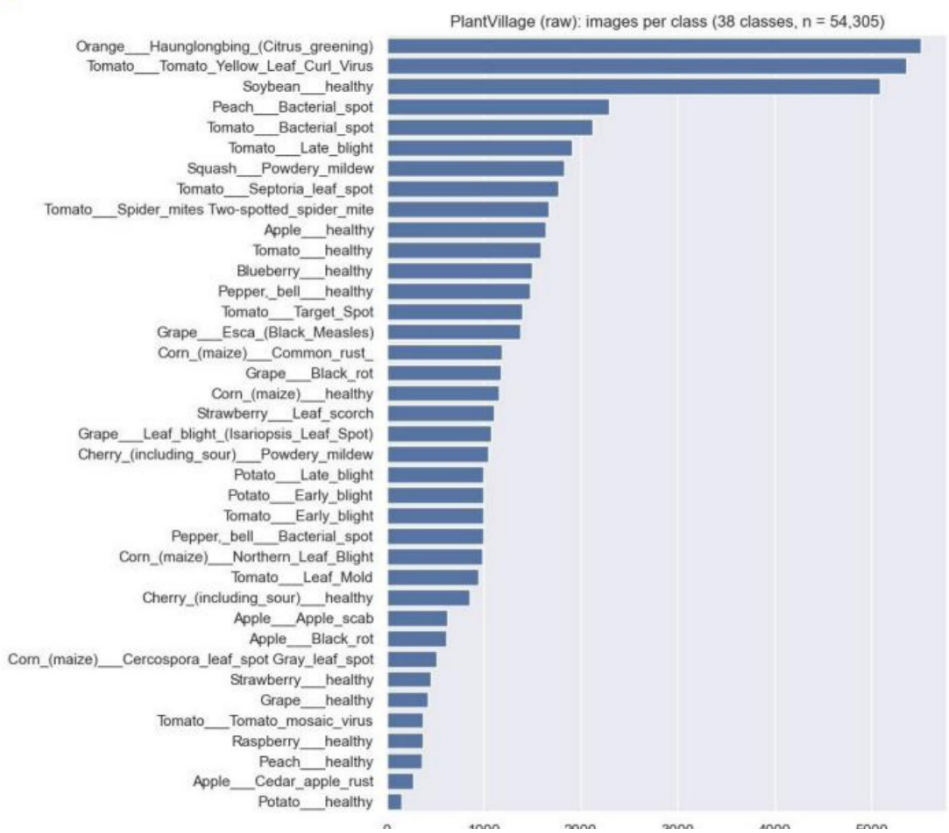

Figure 1 — Images per class (38 classes, n = 54,305).

What it shows. One horizontal bar per class, sorted largest to smallest, makes the imbalance immediate: the largest class, Orange citrus-greening, holds 5,507 images; the smallest, healthy Potato, just 152. A handful of large classes tower over a long tail of small ones.

Statistical validation. A chi-square goodness-of-fit test against an equal-frequency expectation returns $\chi^2 \approx 41{,}874$ with $p \approx 0$. Because the test runs on 54,305 images this rejection is near-certain and says little on its own; the plain imbalance ratio of 36.2 : 1 is the figure that measures severity. The practical consequence is that overall accuracy would mislead, and the imbalance must be corrected during training.

Figure 2 — Healthy vs diseased per crop

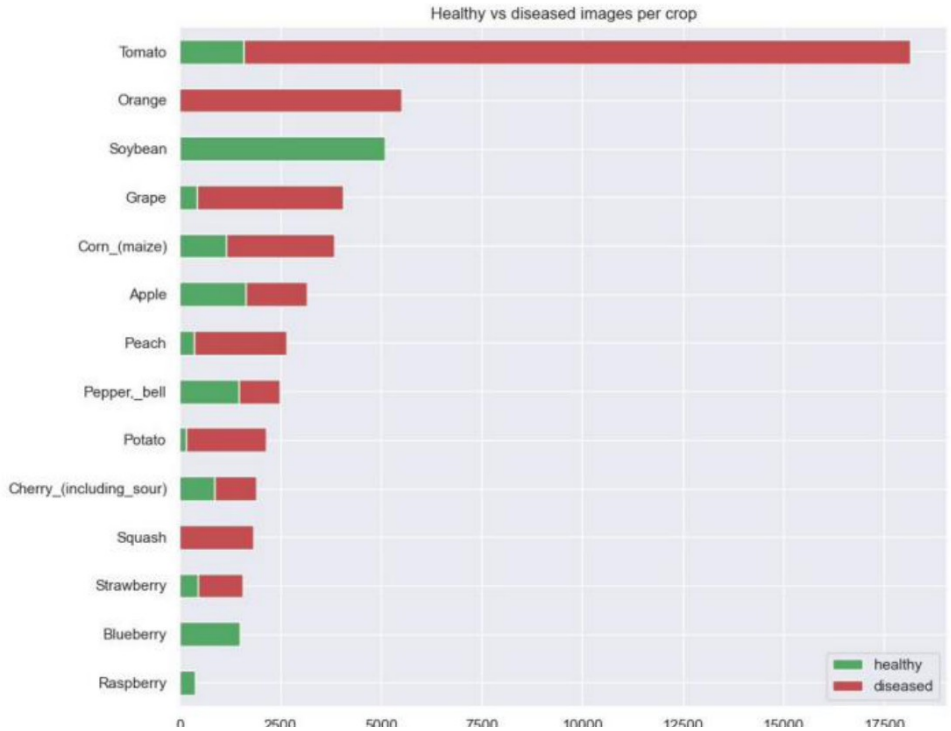

Figure 2 — Healthy (green) vs diseased (red) images per crop.

What it shows. A stacked bar per crop, split into healthy and diseased and sorted by size, shows how the balance varies across crops. Tomato dominates at roughly a third of all images and is overwhelmingly diseased. Crucially, several crops appear under a single condition only: Blueberry, Raspberry and Soybean are entirely healthy, while Orange and Squash are entirely diseased.

Statistical validation. A chi-square test of independence strongly rejects independence ($\chi^2 \approx 28{,}005$, 13 degrees of freedom, $p \approx 0$), with the smallest expected count near 103 — comfortably above the value-of-5 threshold. Crop and disease status are therefore entangled. The single-condition crops are the decisive detail: a "healthy or diseased?" question cannot be learned for them, which is why the task is framed as flat 38-class prediction rather than a per-crop decision.

Figure 3 — Leaf greenness (healthy vs diseased)

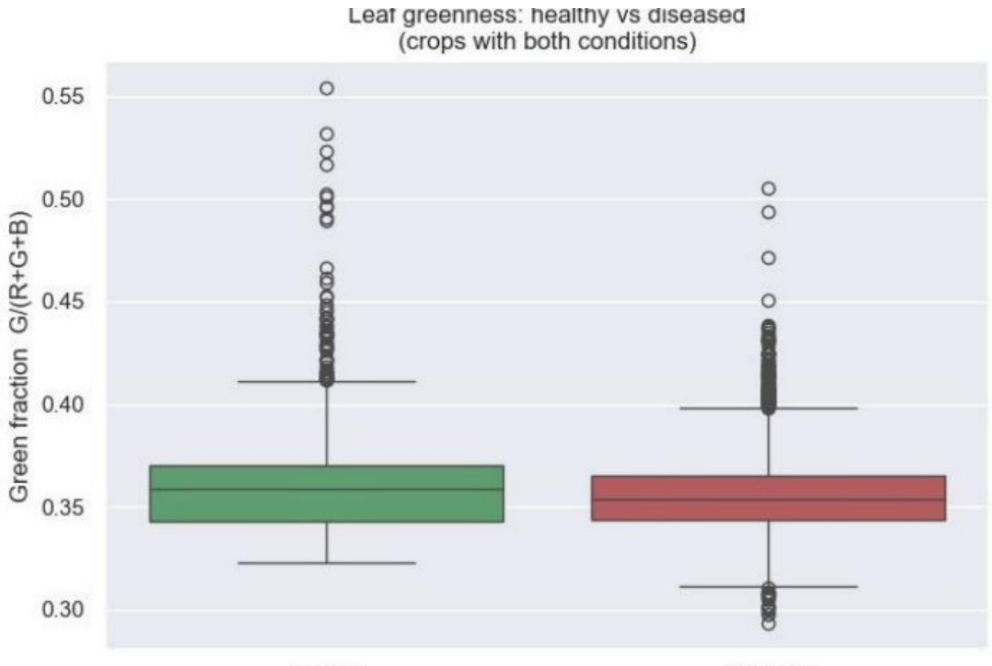

Figure 3 — Green fraction G/(R+G+B): healthy vs diseased (crops with both conditions).

What it shows. To test whether a single global colour summary can separate healthy from diseased leaves, the green fraction — green divided by the total of all three channels, which removes overall brightness — is compared between the two, restricted to crops with both conditions to remove crop as a confounder. The two boxes sit almost on top of one another, medians near 0.358 (healthy) and 0.354 (diseased): visually, almost no separation.

Statistical validation. A one-way ANOVA finds the difference statistically significant ($F \approx 94.8$, $p \approx 2.8e\text{-}22$) but the eta-squared effect size is just 0.011 — status explains only about 1% of the variation in greenness. This is the report's most important result: the difference is real but unimportant, significant only because the test runs on about 8,150 leaves. Disease is local — a few spots on an otherwise green leaf — so a whole-leaf average washes the symptom out. That is the empirical case for a convolutional network over hand-crafted whole-image features.

Figure 4 — Feature correlations

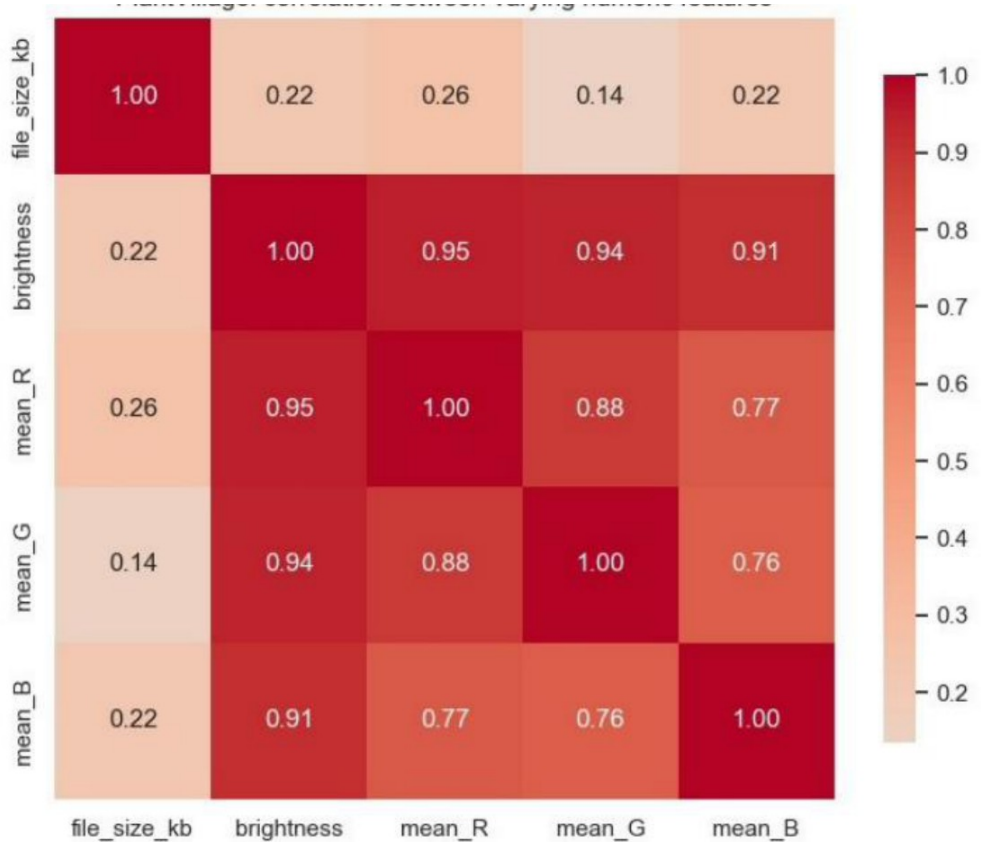

Figure 4 — Correlation between the varying numeric features.

What it shows. A correlation heatmap of the five varying numeric features (file size, brightness and the three colour means). Brightness and the three colour channels are almost perfectly correlated, while file size stands apart.

Statistical validation. Two Pearson correlations confirm the pattern: brightness versus green r = 0.94, and red versus green r = 0.88 ($p \approx 0$). Across the heatmap, brightness tracks each colour channel at 0.91–0.95 and the channels track one another at 0.76–0.88, whereas file size correlates only 0.14–0.26 with the rest. The four colour numbers are effectively one underlying "lightness" dimension measured four ways — far less independent information than their count suggests. This reinforces Figure 3: whole-image colour holds little that separates the classes.

Figure 5 — Healthy vs diseased sample leaves

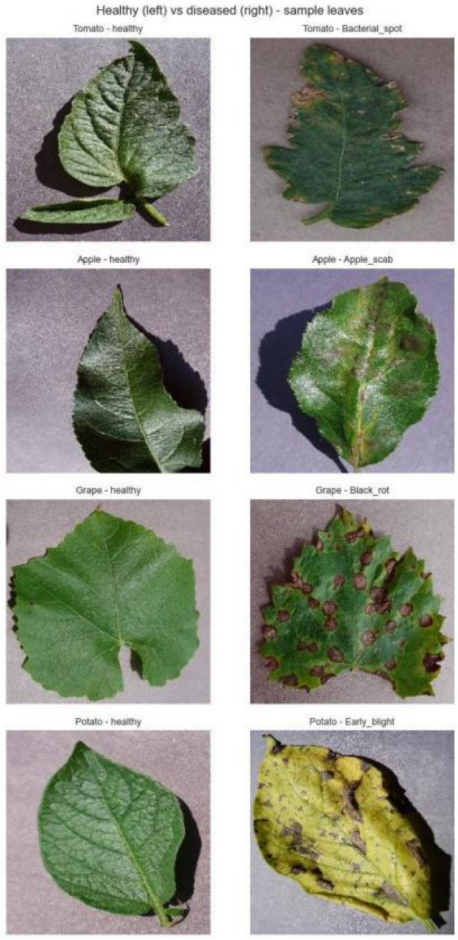

Figure 5 — Healthy (left) vs diseased (right) sample leaves, four crops.

What it confirms. For four crops (Tomato, Apple, Grape, Potato), a healthy leaf sits beside a diseased one. Healthy leaves are an even green; diseased leaves carry localised damage — spots, lesions, ringed patches, yellowing — confined to part of the leaf. Seen directly, the disease signal is plainly local and spatial, exactly as Figures 3 and 4 implied, and a convolutional network is the appropriate model. Every leaf sits on a plain, uniform laboratory background — the clearest sign of the bias discussed next. (This is a qualitative check and carries no statistical test.)

6. Difficulties and biases

The exploration surfaces four issues the modelling must take into account:

- **Severe class imbalance.** At roughly 36:1, the largest class has about thirty-six times the images of the smallest. Left unaddressed, a model favours the common classes and overall accuracy flatters it (Figure 1).

- **Single-condition crops and crop–disease entanglement.** Five crops appear under only one condition, so a per-crop healthy/diseased question is ill-posed for them, and crop is statistically entangled with disease status (Figure 2).

- **A clean-laboratory background.** Every image is a leaf on a uniform studio background. A model can exploit that shortcut instead of learning the disease, inflating its scores and leaving a gap to real-world field photographs (Figure 5).

- **Tomato dominance.** Tomato alone accounts for roughly a third of all images, so the dataset's overall behaviour is heavily influenced by a single crop.

7. Pre-processing and feature engineering

Based on the Step 1 findings, the raw dataset is turned into a modelling-ready pipeline, implemented in a TensorFlow/Keras notebook. Each decision below follows from a specific finding above, and each is tied back to it.

7.1 From findings to a pre-processing pipeline

The pipeline loads the raw color images, builds a stratified and leakage-free train/validation/test split at the file level, resizes the images, and prepares normalisation and augmentation as layers to be embedded in the model. Two transformations a naïve pipeline would bake into the data — normalisation and augmentation — are instead defined as model layers, and the correction of the class imbalance is deferred to training; the reasons are given below. The result is three datasets ready for convolutional-network modelling, together with the layers and the class weights the model will use.

7.2 Loading the images and data cleaning

The color images are loaded with TensorFlow's image_dataset_from_directory, which reads the class structure directly from the folder names and confirms the 38 classes and 54,305 images. The Step 1 audit already established that the data needs no classical cleaning: there are no missing or broken values to impute, the geometry variables are constant so no reshaping is required for consistency, and the images are already three-channel RGB, so no channel standardisation is needed. Pre-processing here is therefore not about repairing the data but about organising it correctly for training.

7.3 A stratified, leakage-free train / validation / test split

The raw set ships without a split, so one is built here — the single most consequential pre-processing step. Step 1 concluded that the split must be stratified (preserving each class's proportion in every part) and must carry no risk of examples leaking across the parts.

The split is therefore performed at the file level. Every image path and its label are enumerated, and scikit-learn's train_test_split is applied twice with stratification — first to separate the test set, then the validation set from the remainder — producing a 70/15/15 split of roughly 38,000 training, 8,150 validation and 8,150 test images. Because the split operates on the lists of files, the three sets are disjoint by construction: no image can appear in more than one.

This deliberately avoids a tempting shortcut — splitting a shuffled, batched dataset by taking and skipping batches. That approach is neither stratified (with 38 classes at 36:1 imbalance, rare classes can be under- or unrepresented in validation and test) nor guaranteed leakage-free (because the underlying stream reshuffles

on each pass, the “skipped” portion is not the complement of the “taken” portion, and the same image can surface in two splits). Both problems would quietly corrupt every downstream metric.

The split is verified programmatically: all 38 classes are confirmed present in each of the three sets, and the sets are asserted to be mutually disjoint. Being built here rather than inherited, it also carries no risk of augmented copies leaking across it — augmentation is applied only after the split and only to the training portion.

7.4 Image resizing

Every image is already a uniform 256×256×3 square, so Step 1 concluded that no resizing is needed for consistency. The pipeline nonetheless downsamples to 128×128 to reduce memory use and speed up training. This is a conscious deviation from that conclusion and is recorded as such: because the disease signal is local (small spots and lesions), reducing resolution discards some of the very detail that carries it. The choice is a compute trade-off, not a data-driven necessity, and it is flagged for revisiting during modelling by comparing 128×128 against the native 256×256 on the validation macro-F1 score.

7.5 Normalisation

Pixel values are scaled from the 0–255 range to 0–1. Rather than applying this to the stored dataset, the normalisation is defined as a Rescaling layer placed first in the model. This keeps the datasets in raw form and guarantees that training, validation and inference all use exactly the same scaling — and it lets the eventual Streamlit application pass raw images straight to the model. A simple linear rescaling (division by 255) is used because neural networks train more stably on small, bounded inputs; no other standardisation is required, as the audit showed the images already uniform.

7.6 Data augmentation

To improve robustness, light augmentation — random horizontal flip, small rotation and small zoom — is applied to the training images. Like normalisation, it is defined as a layer; Keras augmentation layers are active only during training, so it applies to the training data alone and never to validation or test. This augmentation adds within-class variety and acts as regularisation; it is not, by itself, a remedy for class imbalance, because applying the same transformations to every image preserves the class ratios unchanged. Its role in narrowing the laboratory-to-field gap (Section 6) is modest and is revisited as future work.

7.7 Correcting the class imbalance

Step 1 established that the 36:1 imbalance must be corrected during training and that either class weighting or oversampling-with-augmentation is appropriate. The team's chosen approach is class weighting: the loss is weighted inversely to class frequency (weights computed from the training split), so rare diseases are not ignored, combined with the augmentation layer above. Both mechanisms operate at training time, which is efficient on the GPU and adds no images to the dataset; the alternative — physically oversampling minority classes with a pre-processing loop — was rejected because it inflates the dataset and would run on the CPU. For this reason the correction is applied in the modelling step rather than in this notebook. It is recorded here explicitly so that it is not overlooked: the pre-processing does not rebalance the classes; the model must apply the class weights.

7.8 Feature engineering and dimension reduction

No classical feature engineering or dimension reduction is performed, and the exploration explains why. Figures 3 and 4 showed that hand-crafted whole-image features — brightness, the colour means, file size — are highly redundant and barely separate the classes, because the discriminating signal is local rather than global. The appropriate features are therefore the ones a convolutional network learns for itself from the pixels, not summary statistics computed by hand; and reducing the pixel space with a technique such as PCA would discard exactly the local structure the model needs. The "feature engineering" in an image pipeline of this kind is the preparation of the raw pixels — the resizing, normalisation and augmentation above — rather than the construction of tabular features.

7.9 The pre-processed dataset

The pre-processing produces three batched datasets — training, validation and test — of (image, label) pairs, where each image is a 128×128×3 array of floating-point pixel values in the 0–255 range and each label is one of the 38 classes. Alongside them it defines the normalisation and augmentation layers to be embedded in the model and specifies the class weights to be computed from the training labels. The class distribution is preserved across the three splits by stratification, so each set is a faithful miniature of the whole; normalisation shifts the pixel range to 0–1 at the model's input; and, as the audit established, there are no missing values or outliers to remove. The dataset is therefore ready for in-depth analysis and deep-learning modelling.

8. Conclusions and implications for modelling

The exploration and pre-processing together fix the shape of the modelling to come. Each point below follows from a finding or a pre-processing decision above.

- **Treat the task as 38-class classification.** Predicting the full crop___condition label sidesteps the single-condition-crop problem, since a flat label space has no class for the impossible combinations. (A two-stage crop-then-disease design remains a possible alternative to explore.)

- **Use a convolutional neural network.** Because global colour barely separates the classes and the disease signal is local, a CNN that learns local spatial patterns is the appropriate model, not hand-crafted whole-image features.

- **Judge by per-class metrics.** Given the imbalance, the model is measured by macro-F1 and per-class recall rather than accuracy, so the rare diseases are not ignored.

- **Train on a stratified, leakage-free split.** The split is already built and verified; the final performance figure should be reported on the held-out test set.

- **Normalise at the model input; revisit the resolution.** The Rescaling layer normalises to 0–1; the 128×128 versus 256×256 resolution choice should be checked against validation macro-F1.

- **Correct the imbalance with class weights.** Applied in training and combined with augmentation, computed from the training labels.

- **Plan for the laboratory-to-field gap.** Augmentation and, optionally, the segmented rendering can push the model towards the leaf rather than the background, and an honest final assessment would test on real-world field images the laboratory set never contained.

Code files. The work is contained in two notebooks — the Step 1 exploration notebook (Step1_PlantVillage_Exploration) and the Step 2 pre-processing notebook (plant_recognition_preprocess) — with the data audit also exported as DATA_AUDIT_plantvillage.xlsx.